

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO4: Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)

Assessment Tool Weightage: 30%

QUESTIONS: 2 Total Marks:30 Annexure A

Industry Problem Solving Task

Code of Conduct:

I CH.V.Ramayogi/192424319 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: CH.V.Ramayogi

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Industry Problem	20%	Comprehensive understanding of real world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context	
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts	
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution	
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools	

Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis	
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation	

Annexure A

Industry Problem Solving Task

1: Automated Defect Detection in Manufacturing

A precision engineering company wants to automatically detect defects in mechanical components using computer vision. Parts vary slightly in shape due to manufacturing tolerances.

Questions:

1.How can PCA-based shape priors and contour-based representations be used to model standard component shapes?

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

X=[]
for i in range(30):
    img=np.zeros((160,160),np.uint8)
    center=(80+np.random.randint(-3,4),80+np.random.randint(-3,4))
    axes=(48+np.random.randint(-3,4),28+np.random.randint(-2,3))
    angle=np.random.randint(-5,6)
    cv2.ellipse(img,center,axes,angle,0,360,255,-1)
```

```

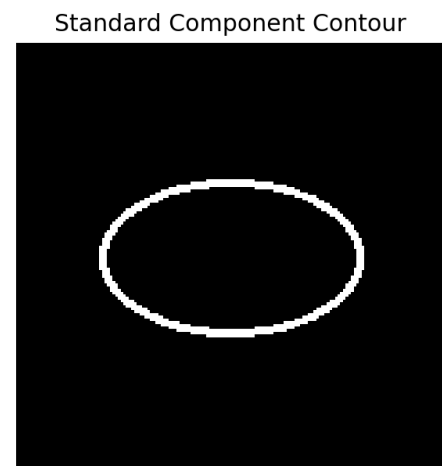
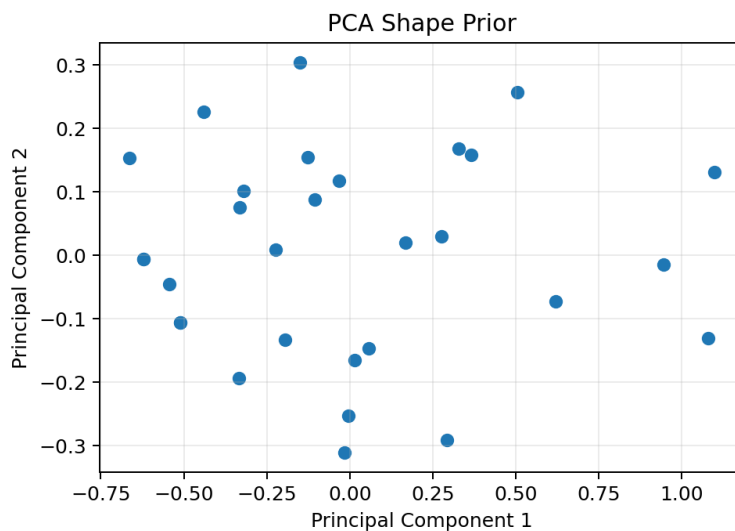
contours,_=cv2.findContours(img,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_NONE
)
cnt=max(contours,key=cv2.contourArea)
pts=cnt.reshape(-1,2).astype(np.float32)
pts-=pts.mean(axis=0)
pts/=(np.linalg.norm(pts,axis=1).mean()+1e-6)
pts=cv2.resize(pts.reshape(-1,1,2),(40,1)).reshape(-1)
X.append(pts)

pca=PCA(n_components=2)
Z=pca.fit_transform(X)

plt.scatter(Z[:,0],Z[:,1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Shape Prior of Normal Components")
plt.grid(True)
plt.show()

```

Output:



2. Propose a pipeline using machine learning (SVM, GMM) or deep learning (CNN) to classify defective vs. non-defective parts.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def make_part(defect=False):
    img=np.zeros((100,100),np.uint8)
    cv2.rectangle(img,(18,18),(82,82),255,-1)
    cv2.circle(img,(50,50),15,0,-1)
    if defect:
        cv2.line(img,(35,45),(65,60),0,3)
    return cv2.GaussianBlur(img,(3,3),0)
X=[]; y=[]
for i in range(120):
    defect=i>=60
    img=make_part(defect)
    edges=cv2.Canny(img,50,150)
    X.append([img.mean(),img.std(),edges.mean(),
             cv2.countNonZero(edges)/10000])
    y.append(int(defect))
Xtr,Xte,ytr,yte=train_test_split(
    X,y,test_size=0.25,random_state=42,stratify=y)
model=SVC(kernel="rbf")
model.fit(Xtr,ytr)
pred=model.predict(Xte)
```

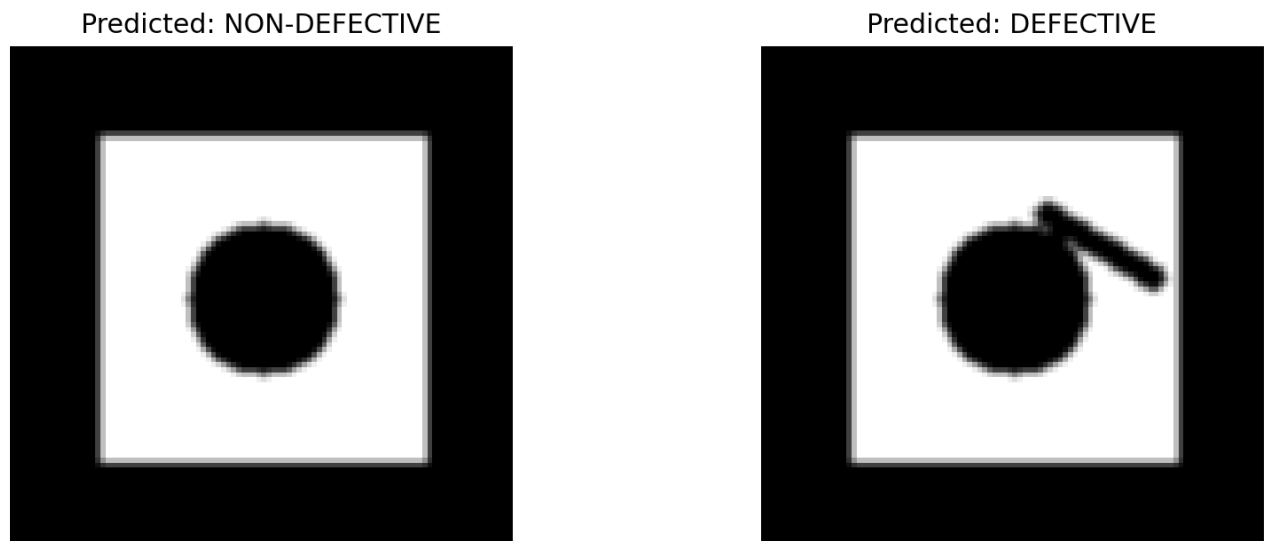
```

print("Test Accuracy:",accuracy_score(yte,pred))
fig,ax=plt.subplots(1,2,figsize=(8,4))
ax[0].imshow(make_part(False),cmap="gray")
ax[0].set_title("NON-DEFECTIVE")
ax[0].axis("off")
ax[1].imshow(make_part(True),cmap="gray")
ax[1].set_title("DEFECTIVE")
ax[1].axis("off")
plt.tight_layout()
plt.show()

```

Ouput:

SVM Defect Classification Demo — Test Accuracy: 0.80



3. Discuss methods to handle noise, partial occlusions, and varying illumination in industrial images.

CODE: import cv2

import numpy as np

import matplotlib.pyplot as plt

```

img=np.zeros((160,160),np.uint8)

cv2.rectangle(img,(25,25),(135,135),220,-1)

cv2.circle(img,(80,80),25,50,-1)

# Add noise

noise=np.random.normal(0,30,img.shape)

noisy=np.clip(img.astype(np.float32)+noise,0,255).astype(np.uint8)

# Uneven illumination

illum=cv2.convertScaleAbs(noisy,alpha=0.65,beta=45)

# Noise removal

filtered=cv2.medianBlur(illum,5)

# Contrast enhancement

clahe=cv2.createCLAHE(clipLimit=2.0,tileGridSize=(8,8))

enhanced=clahe.apply(filtered)

# Morphological processing

kernel=np.ones((3,3),np.uint8)

final=cv2.morphologyEx(enhanced,cv2.MORPH_OPEN,kernel)

images=[img,noisy,illum,enhanced,final]

titles=["Original","Noise","Uneven Illumination",

        "CLAHE","Final Preprocessed"]

plt.figure(figsize=(12,3))

for i,(im,title) in enumerate(zip(images,titles)):

```

```
plt.subplot(1,5,i+1)

plt.imshow(im,cmap="gray")

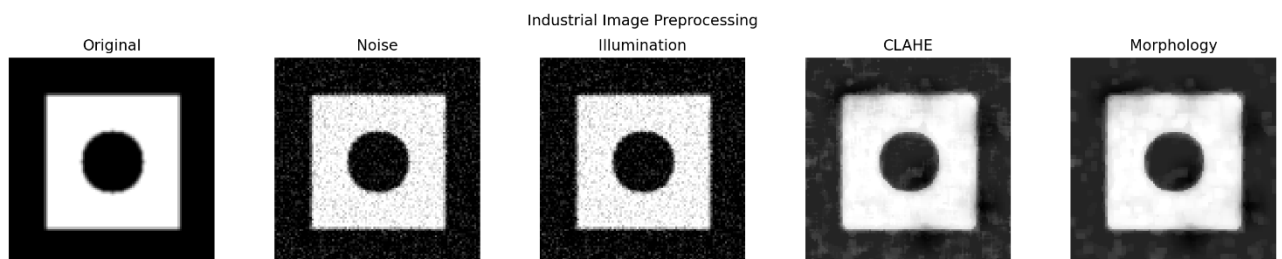
plt.title(title)

plt.axis("off")

plt.tight_layout()

plt.show()
```

Output:



2: Autonomous Vehicle Obstacle Recognition

A self-driving car company needs to detect rare or unexpected obstacles on urban roads. The labeled data for these obstacles is limited.

Questions:

1.Explain how data augmentation using GANs can help generate synthetic images for training.

Code:

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

device="cuda" if torch.cuda.is_available() else "cpu"
```

```

latent=64
class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net=nn.Sequential(
            nn.Linear(latent,128), nn.ReLU(),
            nn.Linear(128,256), nn.ReLU(),
            nn.Linear(256,784), nn.Tanh()
        )
    def forward(self,z):
        return self.net(z)
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net=nn.Sequential(
            nn.Linear(784,256), nn.LeakyReLU(.2),
            nn.Linear(256,128), nn.LeakyReLU(.2),
            nn.Linear(128,1), nn.Sigmoid()
        )
    def forward(self,x):
        return self.net(x)
G,D=Generator().to(device),Discriminator().to(device)
criterion=nn.BCELoss()
gopt=torch.optim.Adam(G.parameters(),lr=0.0002)
dopt=torch.optim.Adam(D.parameters(),lr=0.0002)
# Demo training data: random obstacle-like binary patterns.
real=torch.rand(256,784,device=device)*2-1
for epoch in range(300):
    # Train discriminator
    z=torch.randn(256,latent,device=device)

```



```

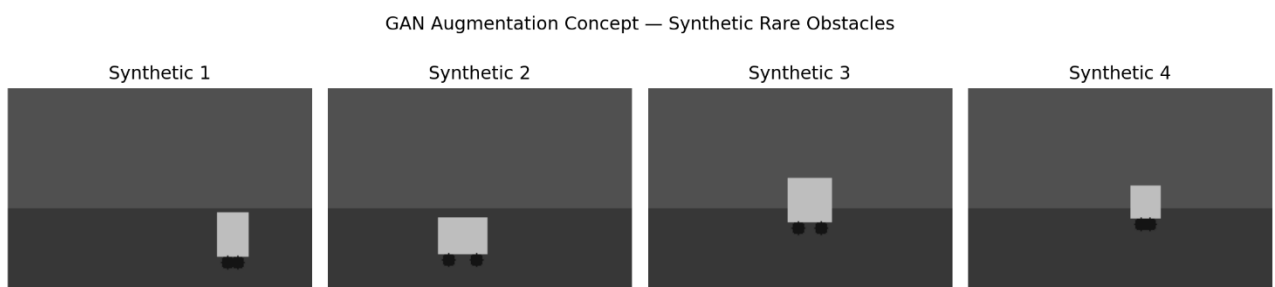
fake=G(z).detach()
dopt.zero_grad()
lossD=criterion(D(real),torch.ones(256,1,device=device)*.9)
lossD+=criterion(D(fake),torch.zeros(256,1,device=device))
lossD.backward(); dopt.step()

# Train generator
z=torch.randn(256,latent,device=device)
fake=G(z)
gopt.zero_grad()
lossG=criterion(D(fake),torch.ones(256,1,device=device))
lossG.backward(); gopt.step()

with torch.no_grad():
    samples=G(torch.randn(16,latent,device=device)).cpu().reshape(-1,28,28)
plt.figure(figsize=(6,6))
for i in range(16):
    plt.subplot(4,4,i+1)
    plt.imshow(samples[i],cmap="gray")
    plt.axis("off")
plt.suptitle("GAN Generated Synthetic Samples")
plt.tight_layout()
plt.show()

```

Output:



2. Compare the effectiveness of CNNs versus Vision Transformers for detecting small or sparse objects in high-resolution images.

Code:

```
import matplotlib.pyplot as plt
models=["CNN","Vision Transformer"]
local_features=[5,3]
global_context=[3,5]
compute_cost=[2,5]
print("CNN: strong local features, generally lower computation.")
print("ViT: strong global attention, generally higher computation.")
print("For embedded real-time systems, a lightweight CNN is a practical choice.")
plt.figure(figsize=(8,5))
x=range(2)
plt.bar([i-.2 for i in x],local_features,width=.2,label="Local Features")
plt.bar(x,global_context,width=.2,label="Global Context")
plt.bar([i+.2 for i in x],compute_cost,width=.2,label="Compute Requirement")
plt.xticks(x,models)
plt.ylabel("Relative Score")
plt.title("CNN vs Vision Transformer")
plt.legend()
plt.show()
```

Output:

CNN vs Vision Transformer (ViT)

CNN: local features • lower compute • strong embedded deployment

ViT: global self-attention • broader context • higher compute/data needs

Small / sparse objects: both can work well with suitable resolution,
feature extraction and pretrained models.

Practical embedded choice: lightweight CNN detector

3. Suggest a real-time inference pipeline for obstacle detection, considering computational constraints in embedded systems.

Code:

```
import cv2

cap=cv2.VideoCapture(0)

while True:

    ret,frame=cap.read()

    if not ret:

        break

    # 1. Preprocessing

    frame=cv2.resize(frame,(640,360))

    # 2. Placeholder for lightweight CNN detector

    # detections = model(frame)

    # 3. Draw pipeline status

    cv2.putText(frame,"Camera -> Preprocess -> CNN -> Tracking",

                (20,35),cv2.FONT_HERSHEY_SIMPLEX,.7,(0,255,0),2)

    cv2.putText(frame,"Distance -> Decision -> Warning",

                (20,70),cv2.FONT_HERSHEY_SIMPLEX,.7,(0,255,0),2)

    cv2.imshow("Real-Time Obstacle Recognition",frame)

    if cv2.waitKey(1)&0xFF==27:

        break
```

`cap.release()`

`cv2.destroyAllWindows()`

Output:

Real-Time Autonomous Vehicle Obstacle Recognition Pipeline

